

Homework — 1.3.1 Compression, Encryption and Hashing — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [2] Lossless compression (1). Valid reason (1), e.g. program code/source files must be restored *exactly* as they were; a single changed/missing bit could corrupt the installer or stop the code compiling; lossy permanently discards data, which would break these files. (No marks if lossy is chosen.)
2. [2] Encoded output: 4R 3G 2R 4B (accept R4 G3 R2 B4). 1 mark for the correct run values in order (R, G, R, B); 1 mark for the correct counts (4, 3, 2, 4). (Check: $4 + 3 + 2 + 4 = 13$ pixels = length of original row.)
3. [2] There are no runs of repeated values — every pixel differs from its neighbour, so every run has length 1 (1). The encoded output (1R 1G 1R 1G ...) is therefore *larger* than the original, because each pixel now needs a count **and** a value stored (1).
4. [3] Up to 3 from: frequently occurring words/patterns/strings are stored once in a dictionary (1); each occurrence in the text is replaced by a short reference/index into the dictionary (1); the reference is smaller than the pattern it replaces, so the overall file is smaller (1); the dictionary must be stored/transmitted with the compressed data so the references can be looked up during decompression (1). Max 3 — must include the dictionary-availability point for full marks.
5. [3] Bob's public key is made openly available / Alice obtains Bob's public key (1). Alice encrypts the file using Bob's **public** key (1). Only Bob's matching **private** key, which is never shared, can decrypt the file — so an interceptor with the public key still cannot read it (1).
6. (a) [1] Hashing is one-way / cannot be reversed — the original password cannot be recovered/calculated from the stored hash (1).
- (b) [1] The password entered at login is hashed using the same hash function and the result is compared with the stored hash; if they match, the password is accepted (1).

Part B (6 marks)

7. [2] One clear difference, 1 mark per side, e.g.: a compiler translates the *whole* program into object/machine code before execution, producing an executable file (1); an interpreter translates and executes one statement at a time each run, producing no standalone executable (1). (Also accept: compiler reports all errors after compilation vs interpreter halts at the first error; compiled code executes faster.)
8. [2] The **MAR** holds the address of the memory location to be accessed — during fetch, the address of the next instruction copied from the PC (1). The **MDR** holds the data or instruction that has been fetched from (or is to be written to) that memory location (1).

9. [2] The scheduler allocates processor time to processes / decides which process the CPU executes next (1). One aim (1, any one): make best use of CPU time; ensure all processes get a fair share of processor time; maximise throughput; give acceptable response times (accept naming a scheduling algorithm such as round robin as exemplification of how an aim is met).

Total: 14 + 6 = 20 marks